

ONCFM

Organisation nationale de lutte contre le faux-monnayage

Détection automatique de faux billets

Conception et évaluation d'un algorithme de classification

Soutenance, projet 12

Charles Lippens

Parcours Data Analyst · Août 2026

Sommaire

1

Contexte et objectifs

ONCFM, problématique, données

2

Analyse exploratoire

EDA, variables discriminantes

3

Préparation des données

Imputation, ACP, standardisation

4

Modélisation

Quatre algorithmes en compétition

5

Évaluation et choix

Métriques, matrices, modèle final

6

Démonstration

Test sur billets_production.csv

Contexte du projet

L'ONCFM, organisation publique, lutte contre les contrefaçons de billets en euros. Une machine consigne automatiquement six dimensions géométriques pour chaque billet analysé.

1 500

billets historiques

6

dimensions géométriques

4

modèles à comparer

Objectif

Construire un algorithme qui prend en entrée les 6 dimensions d'un billet et produit un verdict binaire (vrai / faux), exposée dans une application utilisable par les opérateurs.

Problématique

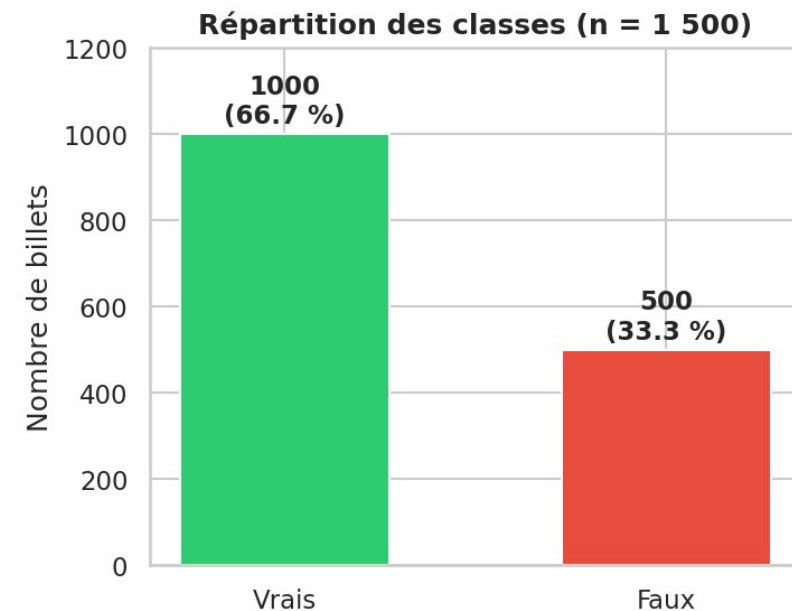
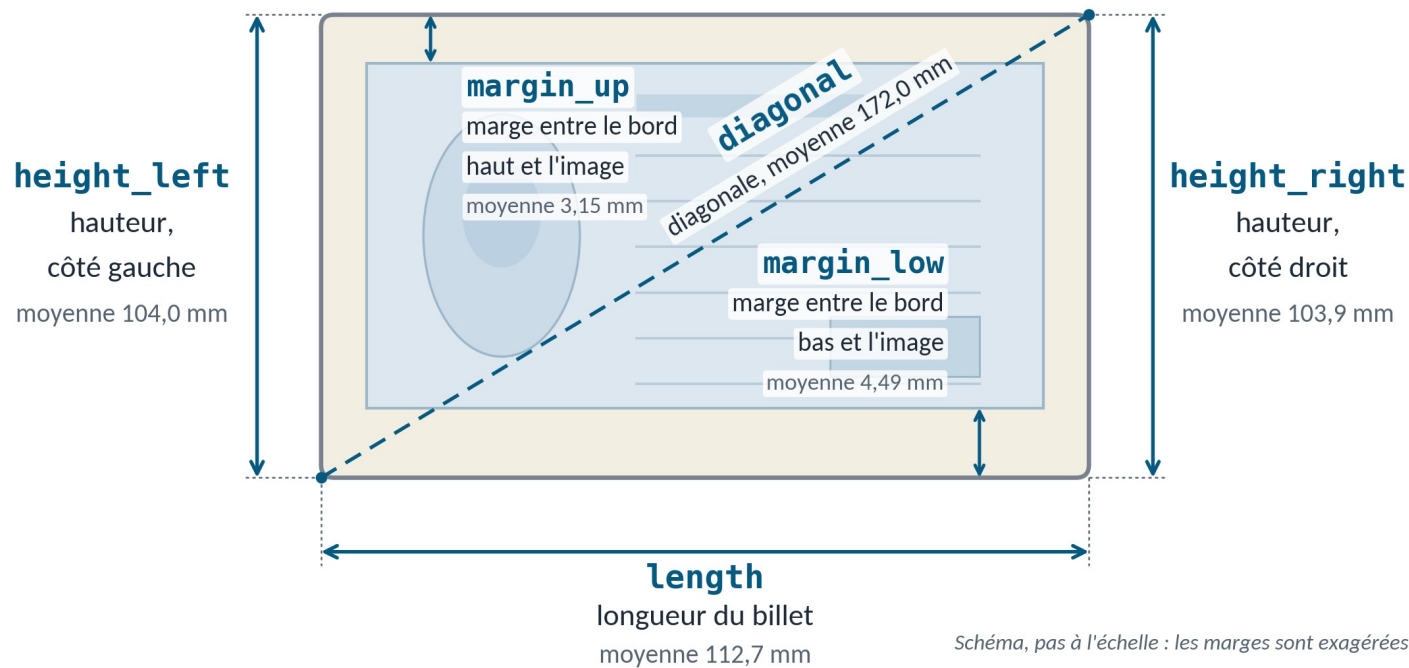
Différencier les vrais et faux billets

→ écarts millimétriques invisibles à l'œil nu mais détectables algorithmiquement.

Délai imposé : 1 mois

Le jeu de données

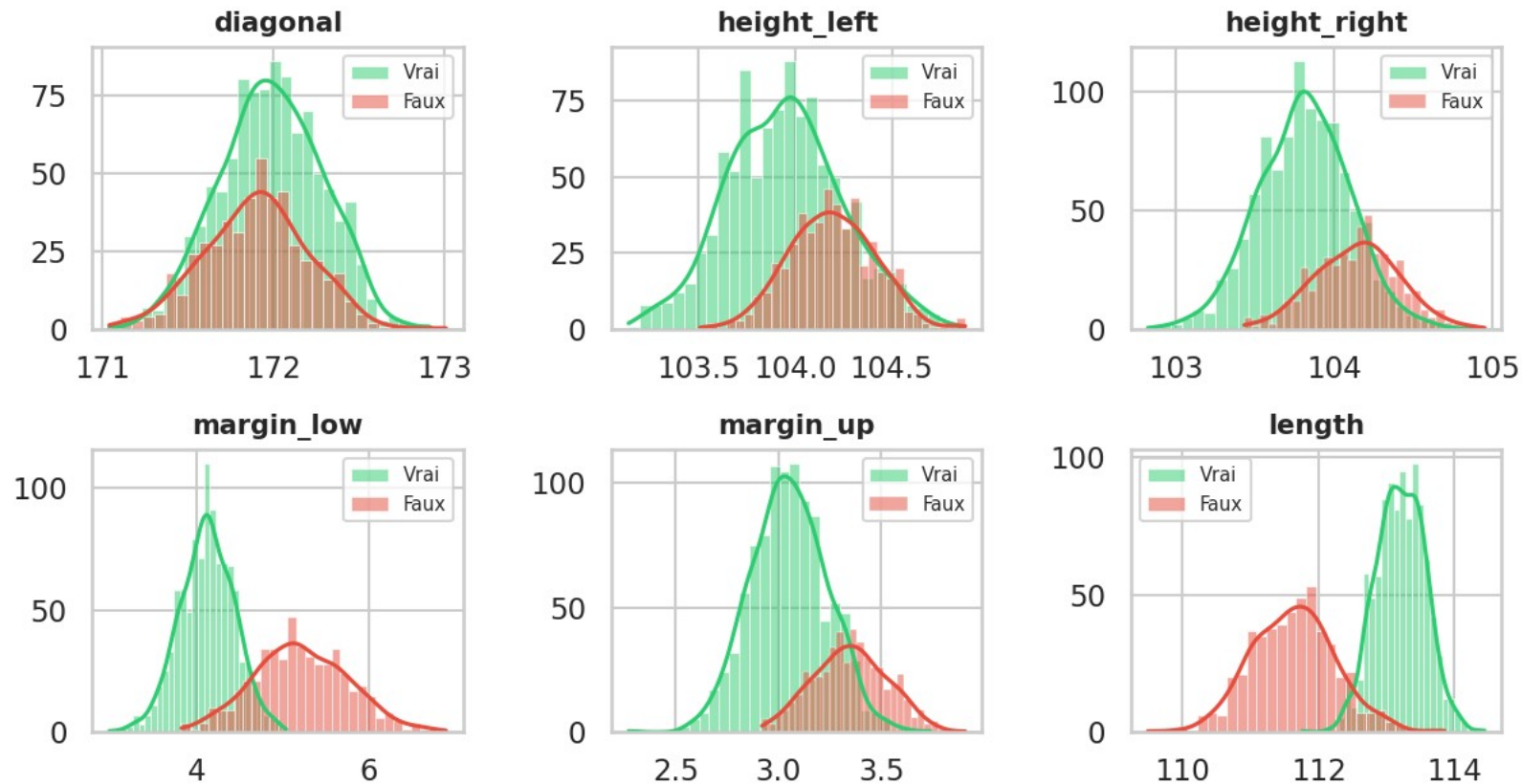
6 variables géométriques (mm)



Cible : is_genuine (booléen) • **Volumétrie** : 1 000 vrais / 500 faux • **Manquants** : 37 sur margin_low (2,47 %) → imputés

EDA : distributions par classe

Distribution des six dimensions par classe



Constats

length

d de Cohen 3,4

margin_low

d de Cohen 2,7

margin_up

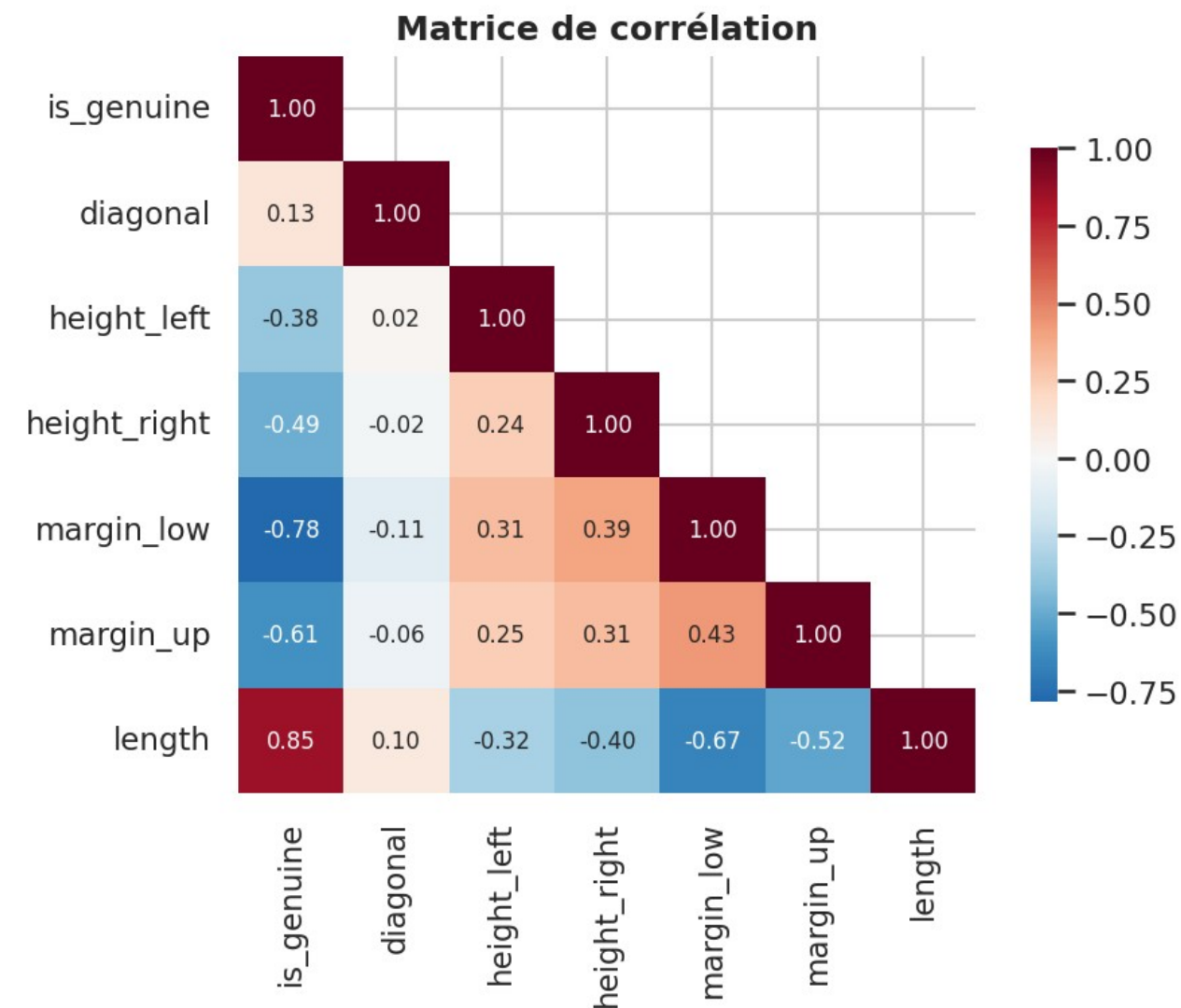
d de Cohen 1,6

diagonal

d de Cohen 0,3

Test de Welch : les 6 variables séparent significativement les classes. La taille d'effet est forte pour 5 d'entre elles, faible pour diagonal.

EDA : corrélations



Corrélations avec is_genuine

length	+0,85
margin_low	-0,78
margin_up	-0,61
height_right	-0,49
height_left	-0,38
diagonal	+0,13

Liaison la plus forte entre variables explicatives : margin_low et length, à -0,67. Facteurs d'inflation de la variance tous inférieurs à 2,2, donc les six variables sont conservées.

Traitement des valeurs manquantes

Diagnostic

37 valeurs manquantes

uniquement sur margin_low (2,47 %)

Variable parmi les plus discriminantes

→ supprimer = perte d'information

Solution retenue

Imputation par régression linéaire OLS

des 5 autres dimensions sur margin_low

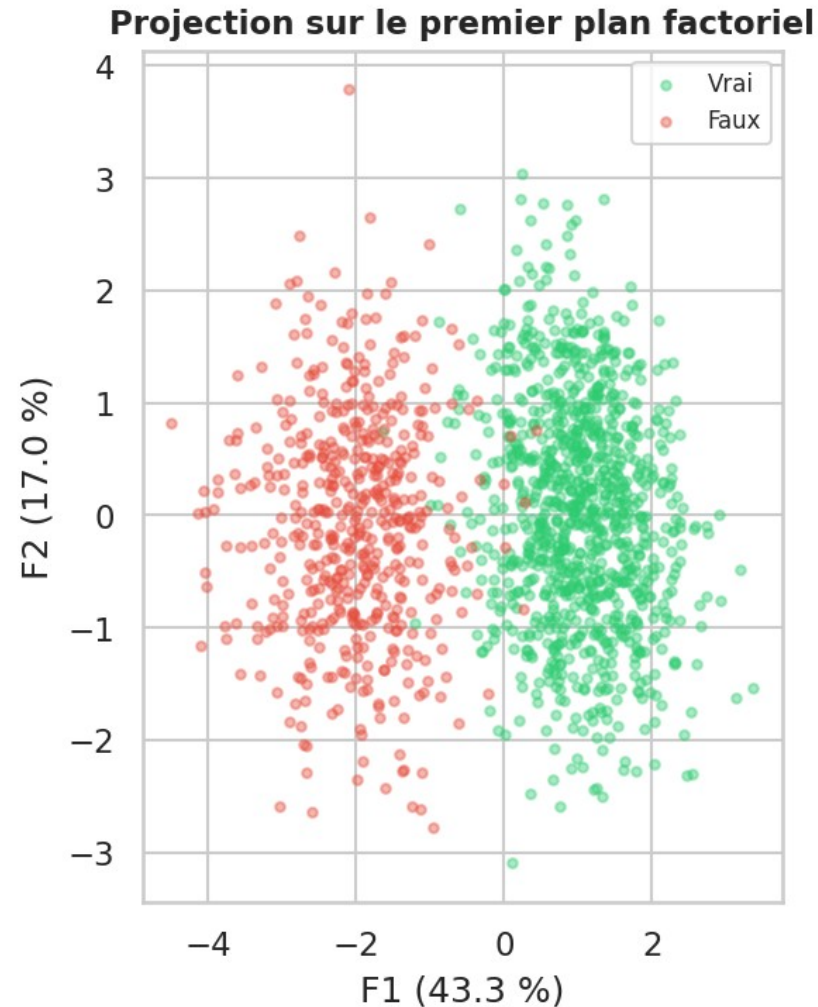
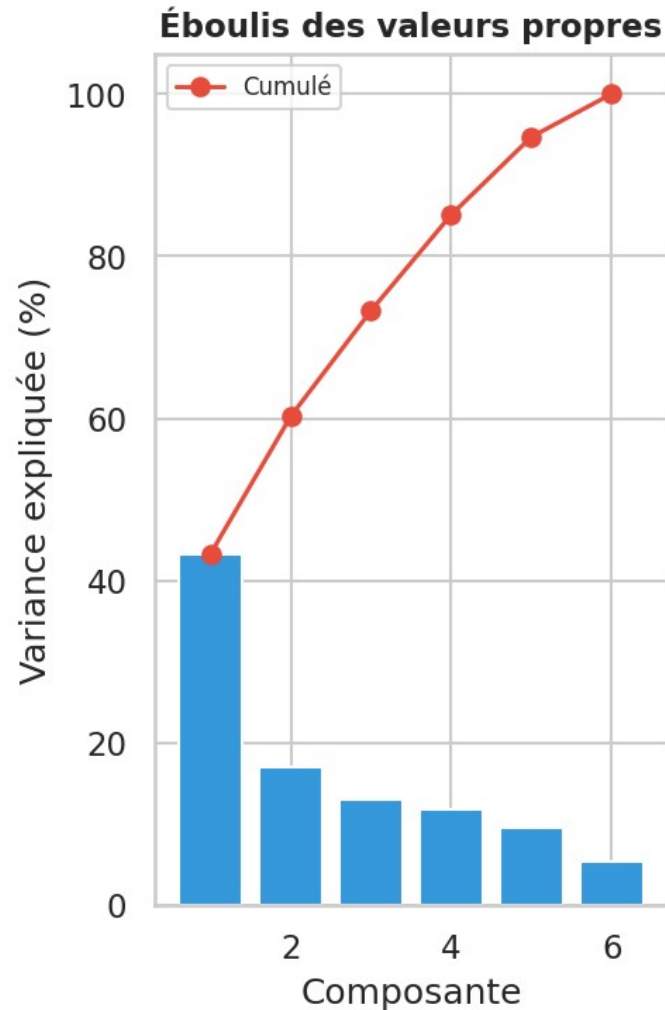
R^2 ajusté 0,476

Résidus centrés, sans structure résiduelle (Q-Q plot et graphe des résidus)

Mise en œuvre

```
model = sm.OLS(y_known, sm.add_constant(X_known)).fit()
df.loc[~mask, 'margin_low'] = model.predict(X_missing)
assert df.isna().sum().sum() == 0    # imputation OK
```

Analyse en composantes principales (ACP)



Lecture de l'ACP

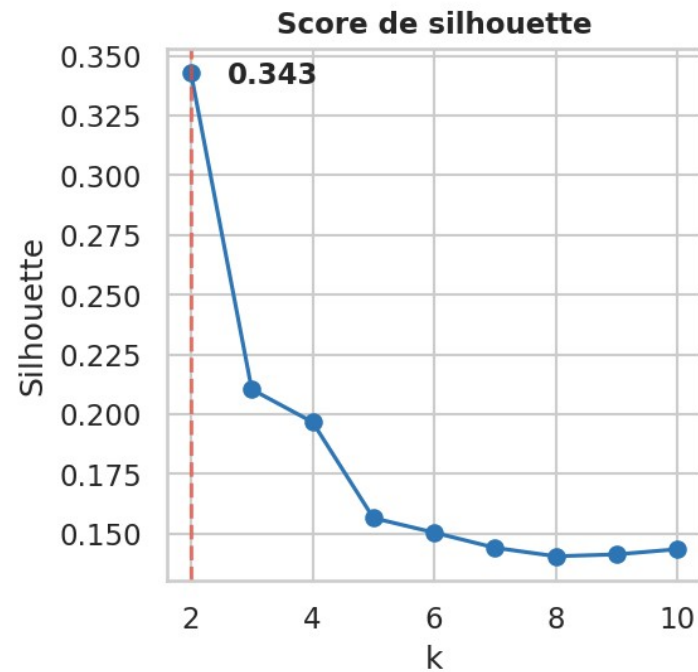
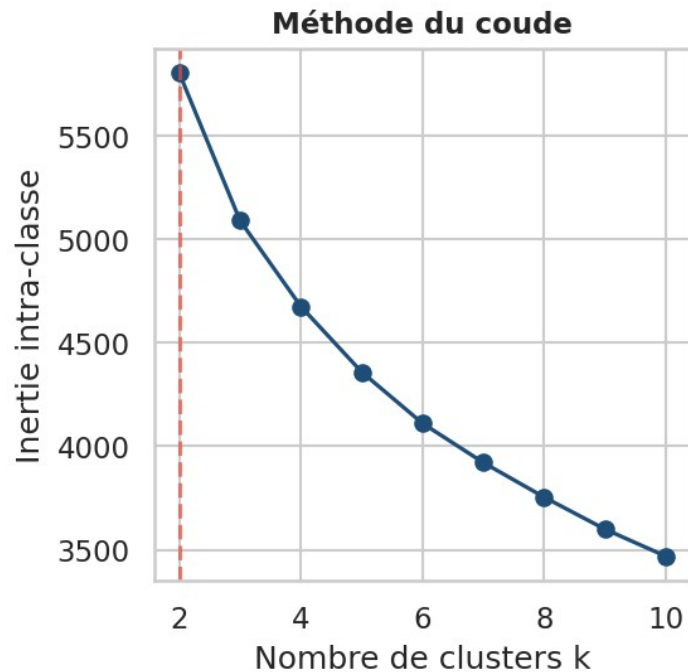
F1 + F2 = 60,2 % de variance

F1 oppose les billets
longs et fins (vrais)
aux billets

courts à grosses marges (faux)

*Les classes sont quasi linéairement
séparables*

Apprentissage non supervisé : K-Means



k optimal = 2

- Coude net à $k = 2$
- Silhouette maximale à $k = 2 : 0,343$
- Cohérent avec l'hypothèse métier (deux populations)

Caractérisation des deux clusters

Cluster A : vrais billets

length 113,20 mm, marges fines. Pureté 98,6 %.

Cluster B : faux billets

length 111,63 mm, marges larges. Pureté 98,0 %.

Méthodologie supervisée

Quatre étapes pour garantir reproductibilité, absence de fuite et stabilité.

1

Split stratifié

80 % train / 20 % test
stratify=y, random_state=42

2

Standardisation

StandardScaler ajusté
uniquement sur le train

3

Validation croisée

5-fold stratifiée pour
choisir les hyperparamètres

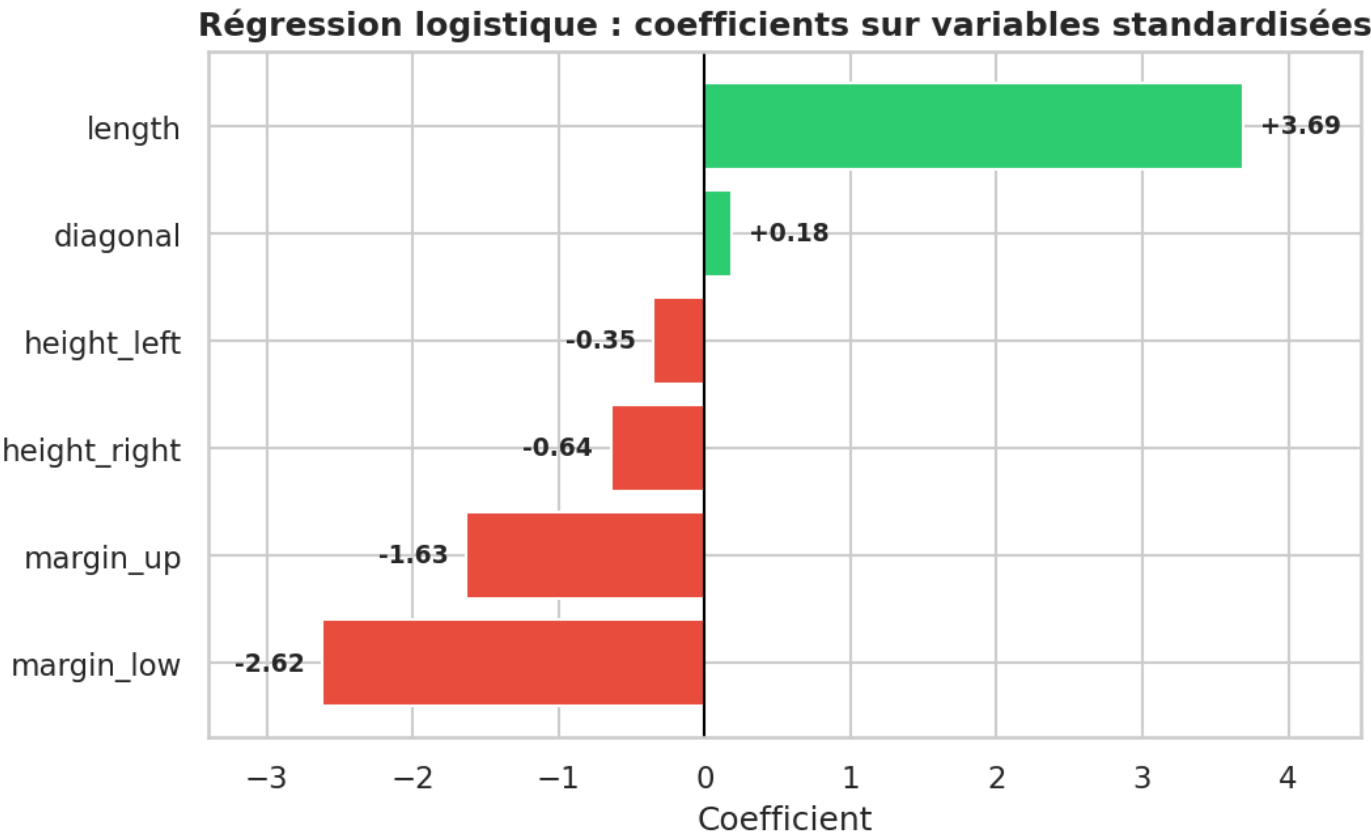
4

Évaluation finale

Métriques sur le test
+ matrice de confusion

Pourquoi ajuster le scaler sur le train uniquement ? Pour éviter la fuite d'information du test vers l'entraînement.

Modèle 1 : régression logistique



Performance test

Accuracy	0,990
Précision	0,990
Rappel	0,995
F1	0,993
ROC-AUC	0,9995

Lecture : length pousse vers « vrai », margin_low/up poussent vers « faux ».

Modèle 2 : K plus proches voisins

Principe

Classification par vote majoritaire des k voisins les plus proches en distance euclidienne sur données standardisées.

Hyperparamètre

k = 13

Sélectionné par GridSearchCV (5-fold) sur F1.

Performance sur le jeu de test

0,987

Accuracy

0,985

Précision

0,995

Rappel

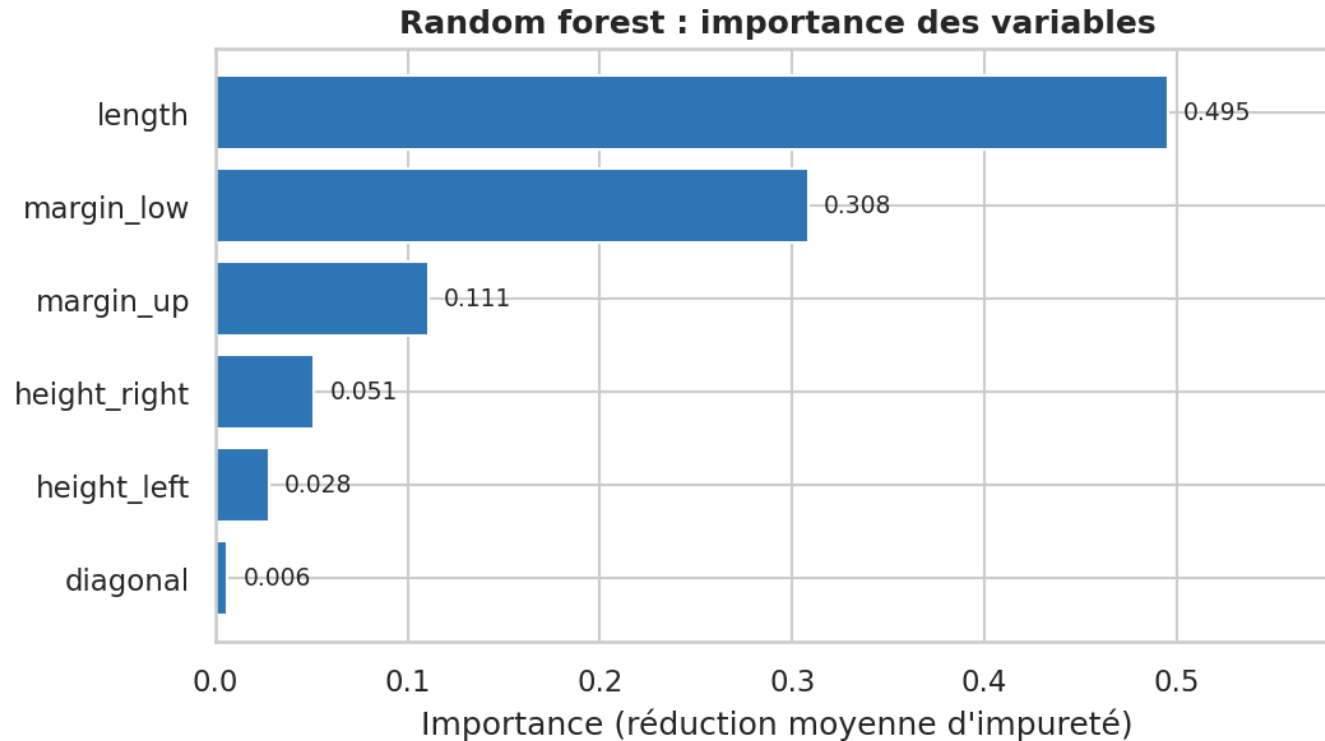
0,990

F1

0,9994

ROC-AUC

Modèle 3 : random forest



Configuration

```
n_estimators = 300  
max_depth = None  
min_samples_leaf = 2  
random_state = 42
```

Pas besoin de standardisation.

Performance sur le test : accuracy 0,990, F1 0,993, ROC-AUC 0,9991. Aucun surapprentissage observé.

Importance des variables cohérente avec la régression logistique : length, margin_low, margin_up dominant.

Modèle 4 : K-Means par les centroïdes

Méthode (4 étapes)

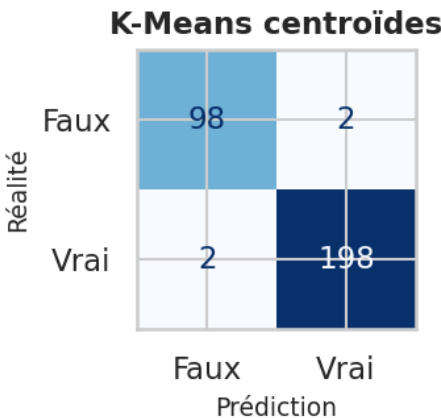
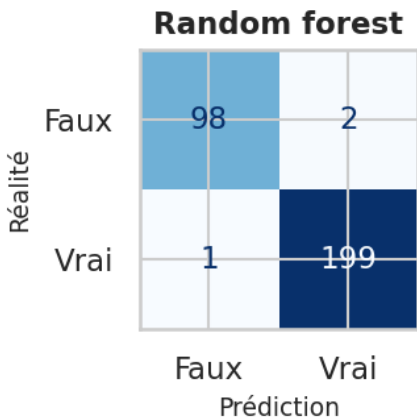
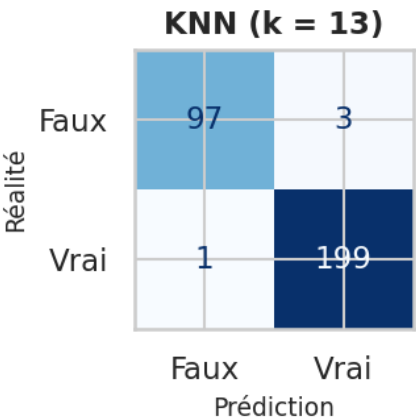
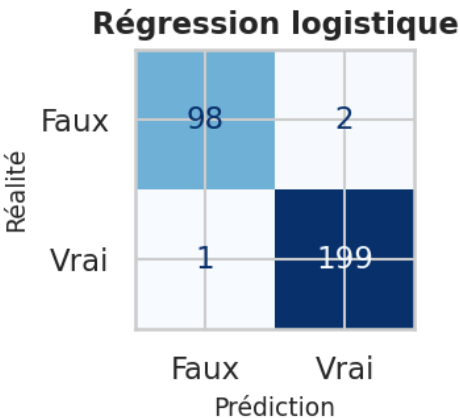
- 1 K-Means($k=2$) entraîné sur $X_{\text{train_std}}$, sans la cible.
- 2 Mapping cluster $\rightarrow$ classe par vote majoritaire des étiquettes.
- 3 Pour un nouveau billet : distance aux centroïdes $\rightarrow$ classe du plus proche.
- 4 Pseudo-probabilité = softmax sur l'inverse des distances.

Limite par construction

L'algorithme n'utilise pas les étiquettes : il ne peut pas, par construction, faire mieux que la séparation géométrique du nuage. Performance attendue inférieure aux supervisés.

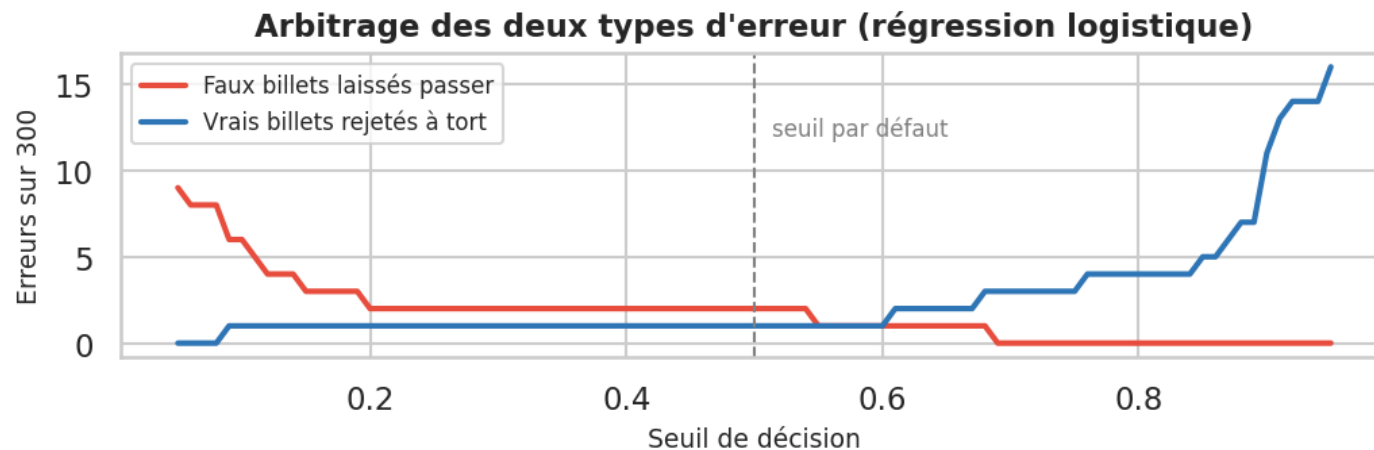
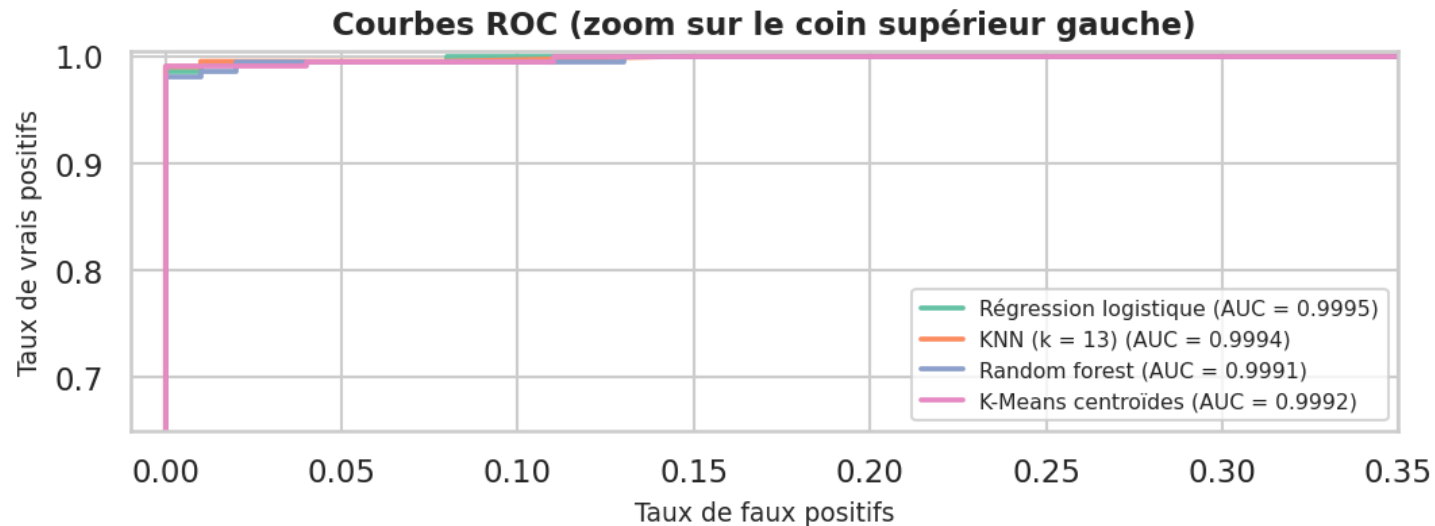
Comparaison des quatre modèles

Matrices de confusion sur le jeu de test (n = 300)



Modèle	Accuracy	Précision	Rappel	F1	ROC-AUC
Régression logistique	0,990	0,990	0,995	0,993	0,9995
KNN (k = 13)	0,987	0,985	0,995	0,990	0,9994
Random forest	0,990	0,990	0,995	0,993	0,9991
K-Means centroïdes	0,987	0,990	0,990	0,990	0,9992

Courbes ROC et arbitrage du seuil



Lecture

En haut : les quatre courbes ROC, zoomées sur le coin supérieur gauche. Toutes dépassent une AUC de 0,999.

En bas : l'effet du seuil de décision sur les deux types d'erreur. Entre 0,3 et 0,5 rien ne bouge, les probabilités sont très tranchées.

Porter le seuil à 0,69 intercepte 100 % des faux du jeu de test, au prix de 2 contrôles manuels de plus. Le levier existe, l'arbitrage revient à l'ONCFM.

Choix final : la régression logistique

MODÈLE RETENU

Régression logistique

Pipeline final :
`StandardScaler + LogisticRegression(max_iter=1000)`

Sauvegarde joblib

```
joblib.dump(pipe,  
'modele_detection_billets.joblib')
```

→ pipeline complet (scaler + modèle) sauvegardé : aucune dérive de prétraitement en production.

Performance

F1 0,993 et ROC-AUC 0,9995, au niveau du random forest.

Interprétabilité

Coefficients lisibles : longueur ↑ ⇒ vrai, marges ↑ ⇒ faux.

Stabilité

Validation croisée 5-fold : F1 0,9945, écart-type 0,0024.

Coût

Un produit scalaire de dimension 6 par billet, adapté à un usage industriel.

Démonstration sur billets_production.csv

Commande lancée dans un terminal, puis tout se passe dans le navigateur

```
streamlit run Lippens_Charles_2_Application_082026.py
```

id	prediction	probabilite_vrai	verdict
A_1	0	0,0011	FAUX BILLET
A_2	0	0,0001	FAUX BILLET
A_3	0	0,0003	FAUX BILLET
A_4	1	0,9794	VRAI BILLET
A_5	1	0,9999	VRAI BILLET

Dans la page : dépôt du fichier CSV, meilleur modèle sélectionné automatiquement, verdicts et probabilités, graphiques (probabilités par billet et plan factoriel), comparaison des quatre modèles, choix manuel du modèle, seuil réglable, résultats téléchargeables. L'application terminal, livrée dans le même dépôt, rend les mêmes verdicts.

3 faux billets détectés et 2 vrais validés, avec des probabilités très tranchées.

Limites et perspectives

Limites identifiées

1 500 billets seulement : jeu d'entraînement modeste, et dérive possible si les contrefaçons évoluent.

Modèle linéaire : les éventuels effets non linéaires ne sont pas capturés par la régression logistique.

Probabilités non calibrées : elles ordonnent correctement les billets mais ne se lisent pas comme des fréquences.

Pistes d'amélioration

Calibration (Platt scaling, isotonic) pour des probabilités fiables.

Détection d'anomalies (Isolation Forest) pour identifier les billets atypiques.

Monitoring de dérive et ré-entraînement périodique trimestriel.

Industrialisation via FastAPI + Docker pour une intégration cloud.

Conclusion

1

Modèle robuste

Régression logistique : $F1 = 0,993$ et $ROC-AUC = 0,9995$ sur le jeu de test.

2

Méthode reproductible

Pipeline scikit-learn complet (StandardScaler + classifieur) sauvegardé via joblib.

3

Application livrable

Application web locale (Streamlit) : fichier déposé dans la page, meilleur modèle automatique, comparaison des quatre modèles, prête à l'emploi pour les opérateurs de l'ONCFM.

Merci. Questions et réponses.